TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

TIỂU LUẬN CUỐI KỲ

Môn học: Trí tuệ nhân tạo

Tên tiểu luận: Xây dựng chương trình tìm kiếm đường đi trên bản đồ sử dụng thuật toán A*, BFS, DFS,UCS, BIDIRECTIONAL SEARCH

Giảng viên: PGS.TS. Hoàng Văn Dũng

Danh sách sinh viên thực hiện

Mã số SV	Họ và tên	Mức độ đóng góp (%)
23110142	Nguyễn Thanh Quang	100%
23110106	Trần Minh Huy	100%
23110107	Đặng Lê Nguyên Hùng	100%

TP. Hồ Chí Minh, tháng 09 năm 2025

LỜI CẢM ƠN

Nhóm em xin gửi lời cảm ơn đến PGS. TS. Hoàng Văn Dũng đã giúp đỡ chúng em trong việc tổng hợp và khai thác thông tin một cách hiệu quả nhất. Cảm ơn Thầy đã tận tình giảng dạy và chỉ bảo trong suốt thời gian diễn ra môn học và giúp nhóm chúng em hoàn thành tốt bài tiểu luận này. Chính nhờ những góp ý của thầy sau mỗi tuần học mà nhóm chúng em khắc phục những hạn chế và cải tiến dự án được tốt hơn.

Tuy đã có nhiều cố gắng nhưng do kiến thức và thời gian nghiên cứu có hạn nên bài tiểu luận của nhóm chúng em không tránh khỏi những sai sót. Do đó, một lần nữa nhóm em rất mong nhận được sự góp ý của Thầy để chúng em có điều kiện hoàn thiện hơn kiến thức của mình.

Cuối lời, em xin chúc thầy thật nhiều sức khỏe và gặt hái được nhiều thành công hơn nữa trong sự nghiệp "trăm năm trồng người". Một lần nữa em xin chân thành cảm ơn.

MỤC LỤC

DANH MỤC HÌNH.....5
DANH MỤC BẢNG.....6
CHƯƠNG 1: MÔ TẢ VỀ BÀI TOÁN.....7
1.1) Phát biểu bài toán.....7
1.2) Mục đích bài toán.....7
1.3) Yêu cầu cần thực hiện.....7
PHẦN 2: CƠ SỞ LÝ THUYẾT DÙNG ĐỂ THỰC HIỆN PROJECT.....8
2.1) Ngôn ngữ lập trình.....8
2.1.1) Giới thiệu về Python.....8
2.1.2) Đặc điểm của ngôn ngữ.....9
2.2) Môi trường lập trình.....9
2.3) Xây dựng giao diện với Pygame.....10
2.4) Đọc – ghi dữ liệu bản đồ ma trận với python tkinter.....11
PHẦN 3: PHÂN TÍCH VÀ THIẾT KẾ GIẢI PHÁP.....12
3.1) Thuật toán A*.....12
3.1.1) Giới thiệu thuật toán A*.....12
3.1.2) Ý tưởng thuật toán.....12
3.1.3) Mô tả thuật toán.....12
3.2) Thuật toán Breadth First Search (BFS).....16
3.2.1) Giới thiệu thuật toán BFS.....16
3.2.2) Ý tưởng thuật toán.....17
3.2.3) Mô tả thuật toán.....17
3.3) Thuật toán Depth First Search (DFS).....20
3.2.1) Giới thiệu thuật toán DFS.....20
3.2.2) Ý tưởng thuật toán.....20
3.2.3) Mô tả thuật toán.....20

PHẦN 4: THỰC NGHIỆM, ĐÁNH GIÁ, PHÂN TÍCH KẾT QUẢ.....23
4.1) Xây dựng giao diện.....23
4.2) Hướng dẫn thực thi phần mềm.....25
4.3) Kết quả sau khi chạy thuật toán.....26

4.4) So sánh các thuật toán.....27
4.4.1) Sử dụng map..... 27
4.4.2) Sử dụng nhiều map và tính trung bình..... 30
PHẦN 5: PHẦN KẾT LUẬN.....31
5.1) Đánh giá những kết quả đã thực hiện được.....31
5.2) Định hướng phát triển.....31
TÀI LIỆU THAM KHẢO..... 32

DANH MỤC HÌNH

Hình 1. python.org – Trang web chính thức của Python.....8
Hình 2. Giao diện Visual Studio Code.....10
Hình 3. Sơ đồ khối thuật toán A*.....14
Hình 4. Code thuật toán A*.....16
Hình 5. Sơ đồ khối thuật toán BFS.....18
Hình 6. Code thuật toán BFS.....19
Hình 7. Sơ đồ khối thuật toán DFS.....21
Hình 8. Code thuật toán DFS.....22
Hình 9. Giao diện phần mềm khi bắt đầu.....23
Hình 10. Hộp thoại chọn file của tkinter.....25
Hình 11. Chọn thuật toán để bắt đầu thực thi.....26
Hình 12. Thuật toán đã thực hiện tìm đường hoàn tất.....27

DANH MỤC BẢNG

Bảng 1. Thông tin các hằng.....24
Bảng 2. Thông tin các nút chức năng.....24

CHƯƠNG 1: MÔ TẢ VỀ BÀI TOÁN

1.1) Phát biểu bài toán(sửa)

Bài toán tìm đường là bài toán tìm kiếm đường đi giữa hai điểm đầu và cuối nếu có lời giải khả thi. Cụ thể với đề tài này, chúng em sẽ mô tả khu vực tìm kiếm bằng ma trận vuông 2 chiều N x N. Mỗi phần tử trong mảng đại diện cho một ô vuông trên bản đồ. Trên bản đồ xuất hiện các ô không thể đi được, gọi là các nút vật cản, là những ô không thể đi vào.

Bài toán sử dụng ba phương pháp tìm kiếm bao gồm tìm kiếm theo chiều rộng, tìm kiếm, theo chiều sâu và giải thuật A*. Biết rằng ở một nút bất kỳ, nó chỉ có thể đi theo 4 hướng lên, xuống, trái, phải.

1.2) Mục đích bài toán(sửa thêm)

- Tìm hiểu và xây dựng thuật toán tìm kiếm theo chiều rộng (BFS), tìm kiếm theo chiều sâu (DFS) và giải thuật A Star.

- Áp dụng tìm đường đi từ 1 điểm đến 1 điểm trên mặt phẳng bằng cách sử dụng các phương pháp tìm kiếm.

1.3) Yêu cầu cần thực hiện

- Tìm hiểu chi tiết các thuật toán được sử dụng.

- Thiết kế giao diện để mô phỏng thực hiện các thuật toán.

- Thực thi chương trình, so sánh và đánh giá các thuật toán dựa trên thời gian thực hiện và chi phí đường đi từ điểm đầu đến điểm cuối.

PHẦN 2: CƠ SỞ LÝ THUYẾT DÙNG ĐỂ THỰC HIỆN PROJECT

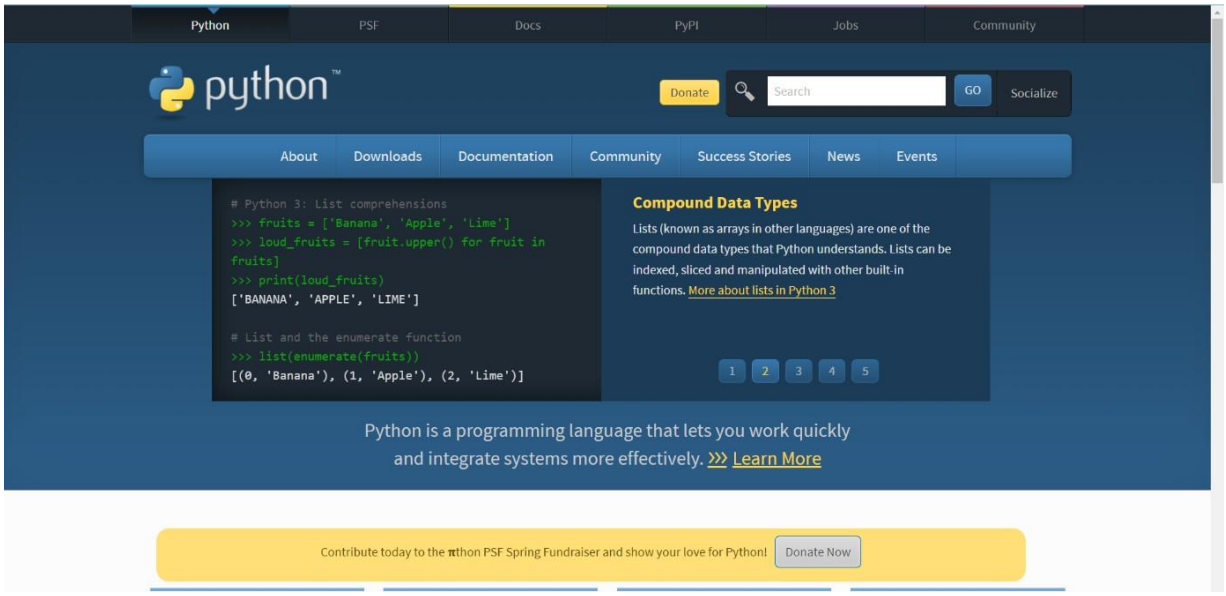
2.1) Ngôn ngữ lập trình

2.1.1) Giới thiệu về Python

Python là một ngôn ngữ lập trình thông dịch (interpreted), hướng đối tượng (object-oriented), và là một ngôn ngữ bậc cao (high-level), ngữ nghĩa động (dynamic semantics). Python hỗ trợ các module và gói (packages), khuyến khích chương trình module hóa và tái sử dụng mã. Trình thông dịch Python và thư viện chuẩn mở rộng có sẵn dưới dạng mã nguồn hoặc dạng nhị phân miễn phí cho tất cả các nền tảng chính và có thể được phân phối tự do.

Python nỗ lực hướng đến một cú pháp đơn giản hơn, gọn gàng hơn trong khi vẫn cho các nhà phát triển lựa chọn phương pháp viết code của họ. Đối lập với khẩu hiệu "có nhiều hơn một cách để làm việc này", triết lý thiết kế của Python lại nằm trong châm ngôn "chỉ nên có một— và tốt nhất là chỉ một—cách rõ ràng để làm việc này".

Thay vì tích hợp hết tất cả các tính năng vào phần cốt lõi, Python được thiết kế để dễ dàng mở rộng (bằng các mô đun). Tính mô đun nhỏ gọn này đã làm cho Python trở nên phổ biến như là một cách thêm các giao diện lập trình được vào các ứng dụng hiện có. Tầm nhìn của Van Rossum (cha đẻ của Python) về một ngôn ngữ có phần lõi nhỏ với một thứ viện chuẩn rộng lớn và một trình thông dịch dễ dàng mở rộng bắt nguồn từ việc ông nản lòng trước ABC, một ngôn ngữ lập trình tán thành hướng tiếp cận ngược lại.



Hình 1. python.org – Trang web chính thức của Python

2.1.2) Đặc điểm của ngôn ngữ

- Python được thông dịch: Python được trình thông dịch xử lý trong thời gian chạy. Bạn không cần phải biên dịch chương trình của mình trước khi thực hiện nó.

- Python có tính tương tác (Interactive): Tại một dấu nhắc Python (command line) bạn có thể tương tác trực tiếp với trình thông dịch để viết chương trình Python.

- Python hỗ trợ kỹ thuật lập trình hướng đối tượng hoặc kỹ thuật lập trình đóng gói mã trong các đối tượng.

- Python là ngôn ngữ linh hoạt và dễ tiếp cận: Python là ngôn ngữ tuyệt vời cho các lập trình viên và hỗ trợ phát triển một loạt các ứng dụng từ xử lý văn bản đơn giản, lập trình web, cho đến lập trình game.

- Python luôn được xếp hạng vào những ngôn ngữ lập trình phổ biến nhất thế giới.

2.2) Môi trường lập trình

Visual Studio Code (VSC) là một trong những trình soạn thảo mã nguồn phổ biến nhất được sử dụng bởi các lập trình viên. Nhanh, nhẹ, hỗ trợ đa nền tảng, nhiều tính năng và là mã nguồn mở chính là những ưu điểm vượt trội khiến VSC ngày càng được ứng dụng rộng rãi. Vì những lý do kể trên, nhóm chúng em đã chọn Visual Studio Code là môi trường lập trình để thực hiện tiểu luận này.

**** Các ưu điểm của Visual Studio Code:***

- Kho lưu trữ an toàn: Đi kèm với sự phát triển của lập trình là nhu cầu về lưu trữ an toàn. Với Visual Studio Code, người dùng có thể hoàn toàn yên tâm vì nó dễ dàng kết nối với Git hoặc bất kỳ kho lưu trữ hiện có nào.

- Hỗ trợ viết Code: một số đoạn code có thể thay đổi chút ít để thuận tiện cho người dùng. Visual Studio Code sẽ đề xuất cho lập trình viên các tùy chọn thay thế nếu có.

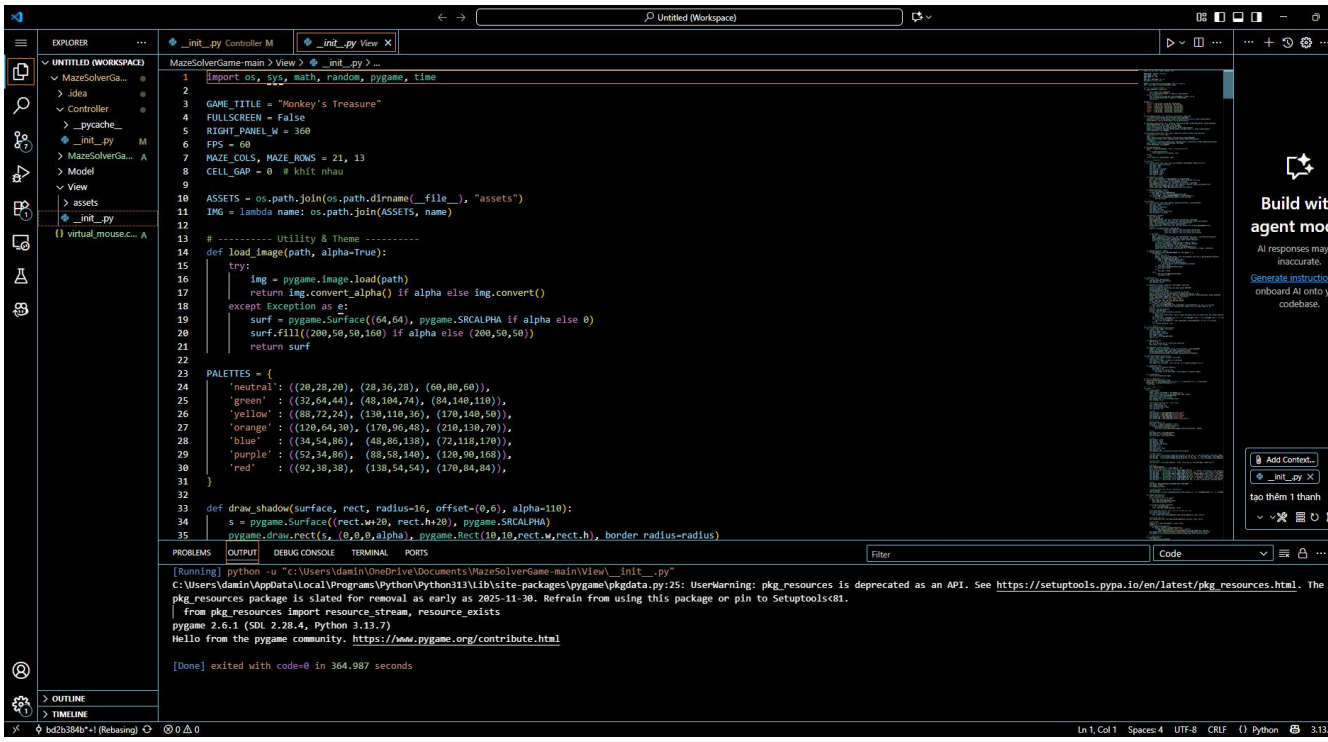
- Màn hình đa nhiệm: Người dùng Visual Studio Code có thể mở cùng lúc nhiều tệp tin và thư mục – việc mở màn hình đa nhiệm giúp ta theo dõi logic code và tính nhất quản của dự án qua các files.

- Giao diện dễ sử dụng, thân thiện với người dùng: Hệ thống các phím tắt đa dạng giúp bạn dễ dàng quan sát cũng như thao tác làm việc nhanh chóng hơn. Từ đó giúp nâng cao hiệu suất và tăng hiệu quả làm việc.

- Khả năng mở rộng và tùy chỉnh:Visual Studio Code cung cấp cho người dùng khả năng tùy chỉnh tuyệt vời như: tùy chỉnh theme, kích thước, font chữ, keyboard shortcut,

coding style, tùy chỉnh tính năng,… một cách linh hoạt. Bên cạnh đó, bạn cũng có thể tùy chỉnh trên từng workshop vô cùng tiện lợi.

- Cài đặt một extension của bên thứ ba trong VSCode: Một trong những tính năng không thể không nhắc đến khi muốn tăng hiệu suất hoạt động của Visual Studio Code đó là sử dụng extension của bên thứ ba. Tương tự theme các bạn có thể dễ dàng tìm được chúng trong Visual Studio Code Marketplace (miễn phí). Extension là một chìa khóa tối đa hóa năng suất của lập trình viên và trong dự án nhóm em sẽ sử dụng Python extension để lập trình trên VS Code.



Hình 2. Giao diện Visual Studio Code

2.3) Xây dựng giao diện với Pygame

Pygame là một bộ mô-đun Python đa nền tảng được thiết kế để viết trò chơi điện tử. Nó bao gồm đồ họa máy tính và thư viện âm thanh được thiết kế để sử dụng với ngôn ngữ lập trình Python.

Pygame sử dụng thư viện Simple DirectMedia Layer (SDL), với mục đích cho phép phát triển trò chơi máy tính trong thời gian thực mà không cần cơ chế bậc thấp của ngôn ngữ lập trình C và các dẫn xuất của nó. Điều này dựa trên giả định rằng các chức năng đắt tiền nhất bên trong trò chơi có thể được trừu tượng hóa khỏi logic trò chơi, do đó có thể sử dụng ngôn ngữ lập trình bậc cao, chẳng hạn như Python, để cấu trúc trò chơi.

Các tính năng khác mà SDL không có bao gồm toán học vectơ, phát hiện va chạm, quản lý đồ họa 2D, hỗ trợ MIDI, camera, thao tác mảng pixel, chuyển đổi, lọc, hỗ trợ phông chữ freetype nâng cao và vẽ.

Trong đề tài này, nhóm chúng em sử dụng Pygame để xây dựng giao diện và mô phỏng chuyển động thuật toán.

Đầu mỗi file, khai báo thư viện pygame qua câu lệnh **'import pygame'**.

2.4) Đọc – ghi dữ liệu bản đồ ma trận với python tkinter

Tkinter là thư viện GUI tiêu chuẩn cho Python. Tkinter trong Python cung cấp một cách nhanh chóng và dễ dàng để tạo các ứng dụng GUI. Tkinter cung cấp giao diện hướng đối tượng cho bộ công cụ Tk GUI.

Với mỗi bản đồ được tạo sẵn qua định dạng file .txt, chương trình sẽ tiến hành đọc file và ánh xạ bản đồ đó lên trên giao diện chương trình. Và để có thể đọc – ghi file .txt để lấy data bản đồ, nhóm chúng em sử dụng thư viện python tkinter.

Trong thư viện tkinter, có tính năng hộp thoại chọn file: filedialog với các hàm như filedialog.askopenfilename() để mở file bản đồ .txt từ hộp thoại và hàm fileopendialog.asksaveasfile() để ghi dữ liệu vào file .txt.

Ta khai báo thư viện tkinter tại file dùng để xử lý dữ liệu có tên file_handling.py, khai báo qua câu lệnh '**import tkinter as tk**' và từ tkinter ta khai báo hộp thoại chọn file '**from tkinter import filedialog'**.

PHẦN 3: PHÂN TÍCH VÀ THIẾT KẾ GIẢI PHÁP

3.1) Thuật toán A*

3.1.1) Giới thiệu thuật toán A*

Thuật toán A* là một thuật toán tìm kiếm trong đồ thị. Thuật toán này tìm một đường đi từ một nút khởi đầu tới một nút đích cho trước (hoặc tới một nút thỏa mãn một điều kiện đích). Thuật toán này sử dụng một "đánh giá heuristic" để xếp loại từng nút theo ước lượng về tuyến đường tốt nhất đi qua nút đó. Thuật toán này duyệt các nút theo thứ tự của đánh giá heuristic này. Do đó, thuật toán A* là một ví dụ của tìm kiếm theo lựa chọn tốt nhất (best-first search).

Thuật toán A* được mô tả lần đầu vào năm 1968 bởi Peter Hart, Nils Nilsson, và Bertram Raphael. Trong bài báo của họ, thuật toán được gọi là thuật toán A; khi sử dụng thuật toán này với một đánh giá heuristic thích hợp sẽ thu được hoạt động tối ưu, do đó mà có tên A*.

3.1.2) Ý tưởng thuật toán

Xét bài toán tìm đường – bài toán mà A* thường được dùng để giải. A* xây dựng tăng dần tất cả các tuyến đường từ điểm xuất phát cho tới khi nó tìm thấy một đường đi chạm tới đích. Tuy nhiên, cũng như tất cả các thuật toán tìm kiếm có thông tin nó chỉ xây dựng các tuyến đường "có vẻ" như dẫn về phía đích.

Để biết những tuyến đường nào có khả năng sẽ dẫn tới đích, A* sử dụng một "đánh giá heuristic" về khoảng cách từ điểm bất kỳ cho tới đích. Trong trường hợp tìm đường đi , đánh giá này có thể là khoảng cách đường chim bay –một cách đánh giá thường dùng cho khoảng cách của đường giao thông.

Điểm khác biệt của A* đối với tìm kiếm theo lựa chọn tốt nhất đó là nó còn tính đến khoảng cách đã đi qua. Điều đó làm cho A* "đầy đủ" và "tối ưu", nghĩa là A* sẽ luôn tìm thấy đường đi nếu tồn tại. A* không đảm bảo sẽ chạy nhanh hơn các thuật toán tìm kiếm đơn giản hơn.

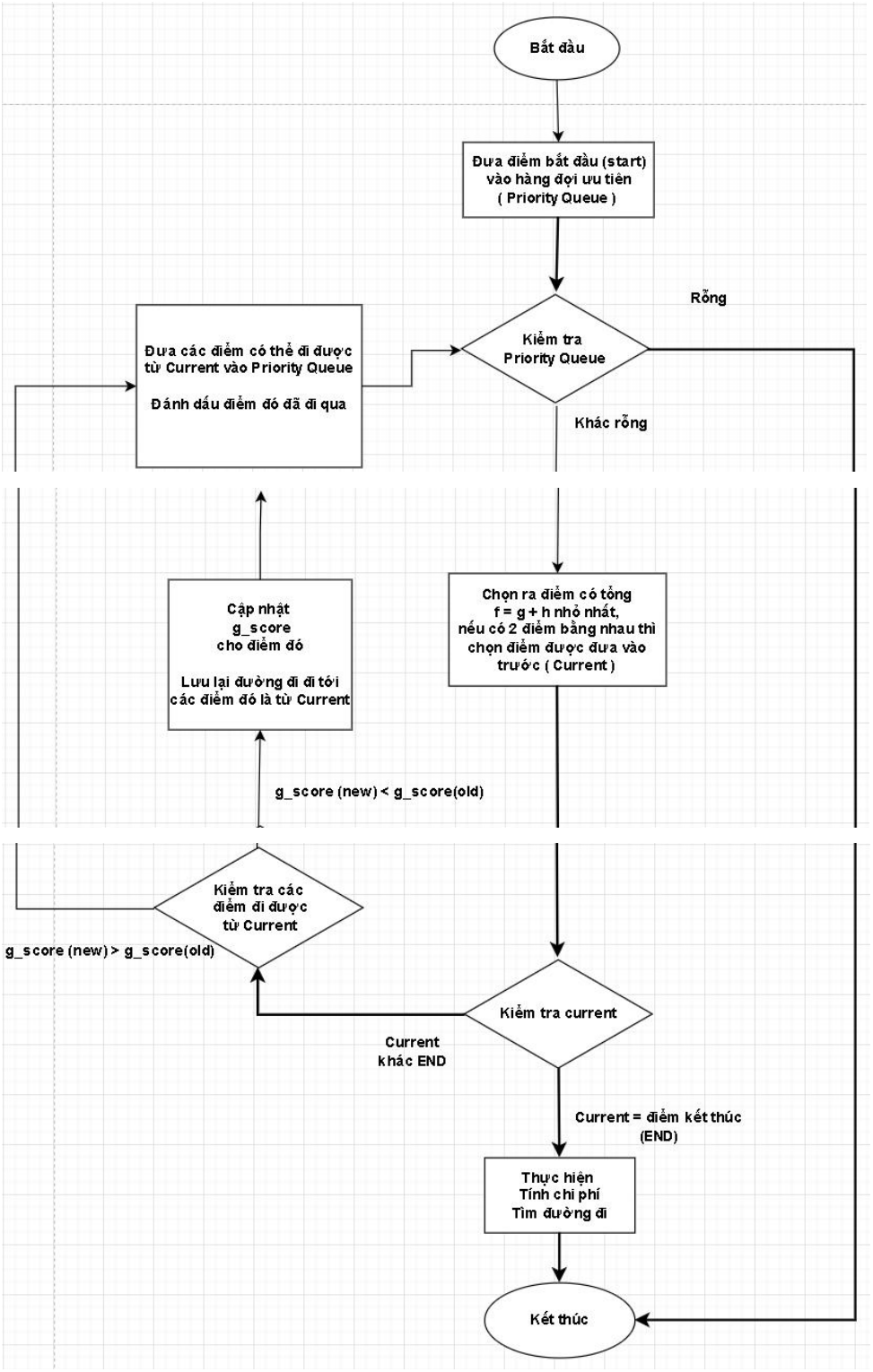
3.1.3) Mô tả thuật toán

A* lưu giữ một tập các lời giải chưa hoàn chỉnh, nghĩa là các đường đi qua đồ thị, bắt đầu từ nút xuất phát. Tập lời giải này được lưu trong một hàng đợi ưu tiên (priority queue). Thứ tự ưu tiên gán cho một đường đi x được quyết định bởi hàm $f(x) = g(x) + h(x)$.

Trong đó, g(x) là chi phí của đường đi cho đến thời điểm hiện tại, nghĩa là tổng trọng số của các cạnh đã đi qua. h(x) là hàm đánh giá heuristic về chi phí nhỏ nhất để đến đích từ x. Ví dụ, nếu "chi phí" được tính là khoảng cách đã đi qua, khoảng cách đường chim bay giữa hai điểm trên một bản đồ là một đánh giá heuristic cho khoảng cách còn phải đi tiếp.

Hàm f(x) có giá trị càng thấp thì độ ưu tiên của x càng cao (do đó có thể sử dụng một cấu trúc heap tối thiểu để cài đặt hàng đợi ưu tiên này).

- **Sơ đồ khối:**



Hình 3. Sơ đồ khối thuật toán A*

- **Code**

```
def A_star(self) -> bool:
    """A* Search - Tìm đường đi tối ưu với heuristic"""
    if not self.start_pos or not self.end_pos:
        return False

    # Priority queue: (f_score, g_score, position)
    heap = [(0, 0, self.start_pos)]
    visited = set()
    came_from = {self.start_pos: None}
    g_score = {self.start_pos: 0}

    while heap:
        f_score, g_score_current, current = heapq.heappop(heap)

        if current in visited:
            continue

        visited.add(current)
        self.nodes_expanded += 1

        if current == self.end_pos:
            self.solution_path = self.reconstruct_path(came_from)
            self.path_length = len(self.solution_path)
            return True
```

```
        for neighbor in self.get_neighbors(*current):
            tentative_g_score = g_score_current + 1

            if neighbor not in visited and (neighbor not in g_score or tentative_g_score < g_score[neighbor]):
                g_score[neighbor] = tentative_g_score
                f_score = tentative_g_score + self.heuristic(neighbor, self.end_pos)
                came_from[neighbor] = current
                heapq.heappush(heap, (f_score, tentative_g_score, neighbor))

                # Đánh dấu ô đã thăm
                if neighbor != self.end_pos:
                    self.maze_grid[neighbor[1]][neighbor[0]].status = 5  # Moved Path

                self.visited_cells.append(neighbor)

    return False
```

Hình 4. Code thuật toán A*

3.2) Thuật toán Breadth First Search (BFS)

3.2.1) Giới thiệu thuật toán BFS

Breadth First Search là một thuật toán tìm kiếm trong đồ thị. BFS lần đầu được phát minh vào năm 1945 bởi Konrad Zuse và không được giới thiệu rộng rãi cho đến năm 1972. Vào năm 1959, thuật toán đã được cải tiến và phát minh lại bởi Edward F. Moore.

3.2.2) Ý tưởng thuật toán

Từ một đỉnh (nút) gốc ban đầu.

- Xác định và lần lượt duyệt các đỉnh kề xung quanh đỉnh gốc vừa xét.

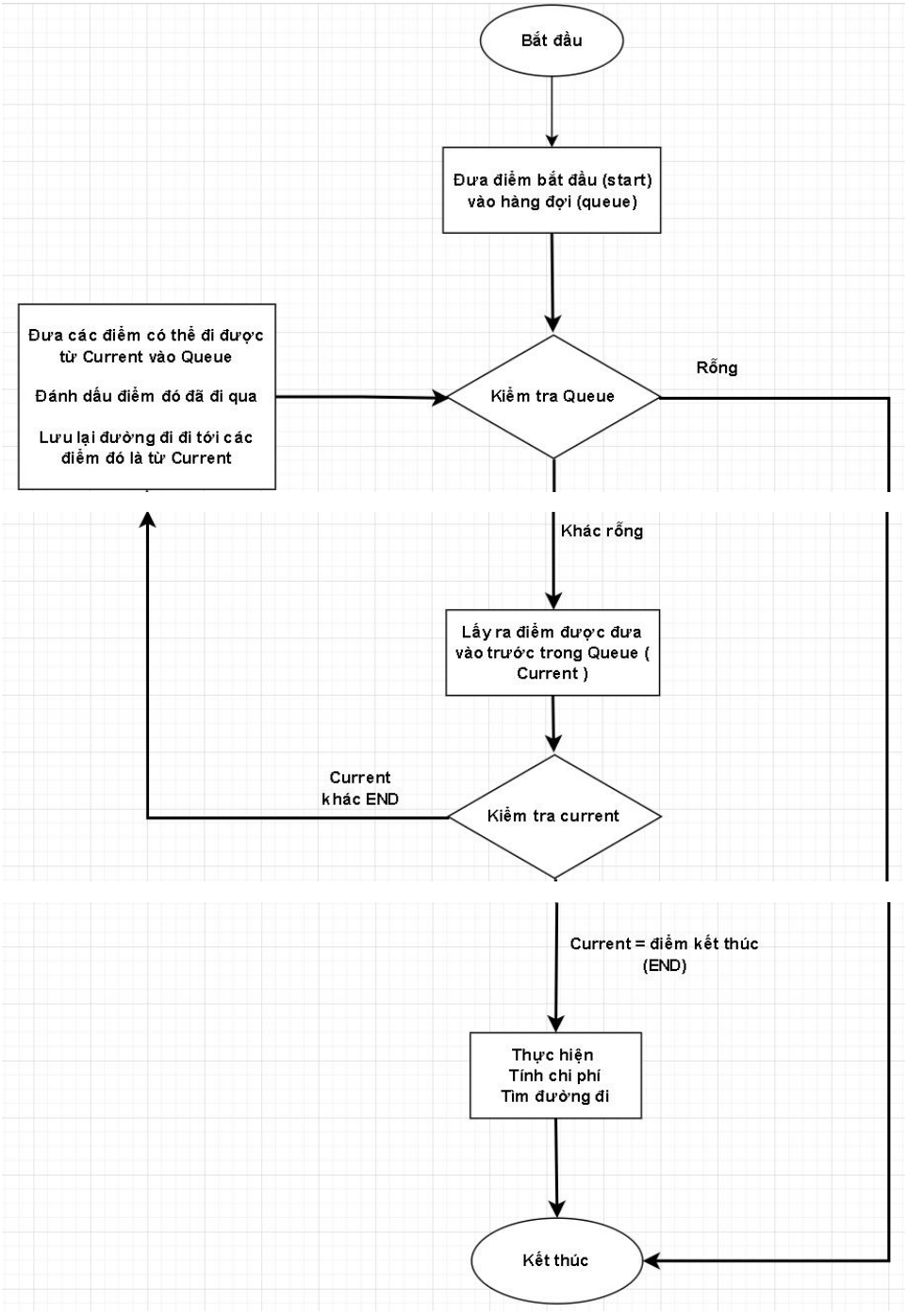
- Tiếp tục quá trình duyệt qua các đỉnh kề đỉnh vừa xét cho đến khi đạt được kết quả cần tìm hoặc duyệt qua tất cả các đỉnh.

3.2.3) Mô tả thuật toán

Thuật toán sử dụng một cấu trúc dữ liệu hàng đợi để lưu trữ thông tin trung gian thu được trong quá trình tìm kiếm:

1. Chọn điểm bắt đầu, chọn ra các đỉnh có thể đi được từ điểm bắt đầu đưa vào hàng đợi (đang hướng tới)
2. Lấy ra đỉnh đầu tiên trong hàng đợi và quan sát nó:
   - Nếu đỉnh này chính là đỉnh đích, dừng quá trình tìm kiếm và trả về kết quả.
   - Nếu không phải thì chèn tất cả các đỉnh kề với đỉnh vừa thăm nhưng chưa được quan sát trước đó vào hàng đợi.
3. Nếu hàng đợi là rỗng, thì tất cả các đỉnh có thể đến được đều đã được quan sát – dừng việc tìm kiếm và trả về "không thấy".
4. Nếu hàng đợi không rỗng thì quay về bước 2.

SƠ ĐỒ KHỐI



Hình 5. Sơ đồ khối thuật toán BFS

CODE:

```
def BFS(self) -> bool:
    """Breadth-First Search - Tìm đường đi ngắn nhất"""
    if not self.start_pos or not self.end_pos:
        return False

    queue = deque([self.start_pos])
    visited = {self.start_pos}
    came_from = {self.start_pos: None}

    while queue:
        current = queue.popleft()
        self.nodes_expanded += 1

        if current == self.end_pos:
            self.solution_path = self.reconstruct_path(came_from)
            self.path_length = len(self.solution_path)
            return True

        for neighbor in self.get_neighbors(*current):
            if neighbor not in visited:
                visited.add(neighbor)
                came_from[neighbor] = current
                queue.append(neighbor)

                # Đánh dấu ô đã thăm
                if neighbor != self.end_pos:
                    self.maze_grid[neighbor[1]][neighbor[0]].status = 5  # Moved Path

                self.visited_cells.append(neighbor)

    return False
```

Hình 6. Code thuật toán BFS

3.3) Thuật toán Depth First Search (DFS)

3.2.1) Giới thiệu thuật toán DFS

Một phiên bản của tìm kiếm theo chiều sâu đã được nhà toán học người Pháp Charles Pierre Trémaux nghiên cứu vào thế kỷ 19 như một chiến lược để giải các mê cung.

3.2.2) Ý tưởng thuật toán

- Từ một đỉnh, ta đi qua sâu vào từng nhánh. Khi đã duyệt hết nhánh của đỉnh, lùi lại để duyệt đỉnh tiếp theo.

- Thuật toán dừng khi đi qua hết tất cả các đỉnh có thể đi qua.

3.2.3) Mô tả thuật toán

- Đây là thuật toán tìm các đỉnh bằng cách duyệt theo chiều sâu.

- Xuất phát từ 1 đỉnh và đi cho đến khi không thể đi tiếp, sau đó đi về lại đỉnh đầu.

Trong quá trình quay lại:

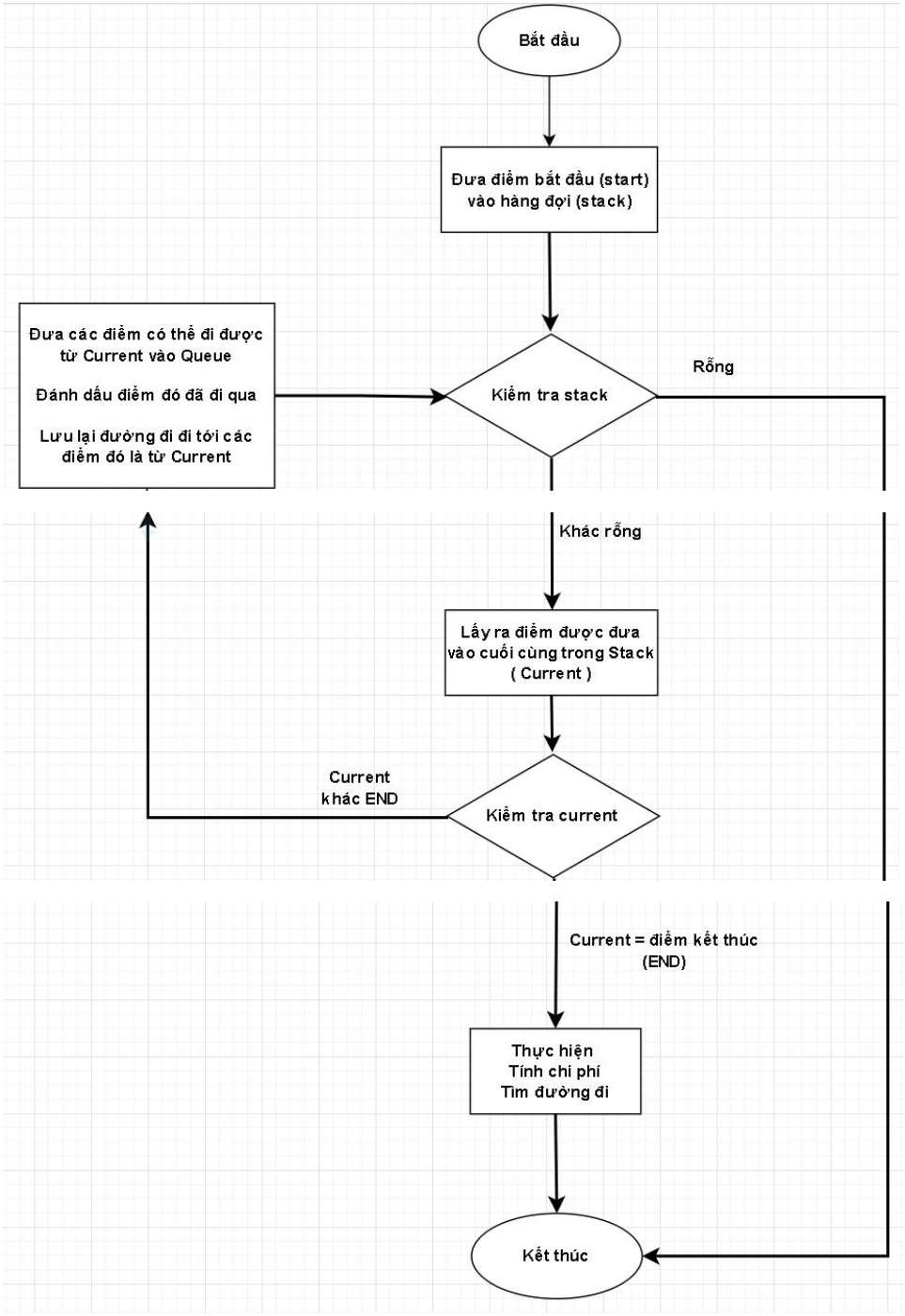
+ Nếu gặp đường đi khác thì đi cho đến khi không đi tiếp được nữa

+ Nếu không tìm ra đường đi nào khác thì ngừng việc tìm kiếm.

Trong quá trình đi đến đỉnh khác, thuật toán sẽ lưu lại đỉnh cha vừa đi qua để khi đi ngược lại từ đỉnh Kết thúc đến đỉnh Xuất phát, ta có thể xem được đường đi từ đỉnh Kết thúc đến đỉnh Bắt Đầu (có thể số lần đi không ít nhất, các bạn có thể tham khảo thuật toán BFS).

– Sở dĩ thuật toán này tìm được đường đi là nhờ vào cơ chế tô màu và lưu đỉnh cha. Quá trình tô màu khiến 1 đỉnh không thể xét 2 lần trở lên và có thể xem được đường đi từ đỉnh Kết Thúc đến đỉnh Xuất phát dựa vào việc lưu đỉnh cha.

SƠ ĐỒ KHỐI:



Hình 7. Sơ đồ khối thuật toán DFS

```python
def DFS(self) -> bool:
    """Depth-First Search - Tìm một đường đi (không nhất thiết ngắn nhất)"""
    if not self.start_pos or not self.end_pos:
        return False

    stack = [self.start_pos]
    visited = {self.start_pos}
    came_from = {self.start_pos: None}

    while stack:
        current = stack.pop()
        self.nodes_expanded += 1

        if current == self.end_pos:
            self.solution_path = self.reconstruct_path(came_from)
            self.path_length = len(self.solution_path)
            return True

        for neighbor in self.get_neighbors(*current):
            if neighbor not in visited:
                visited.add(neighbor)
                came_from[neighbor] = current
                stack.append(neighbor)

                # Đánh dấu ô đã thăm
                if neighbor != self.end_pos:
                    self.maze_grid[neighbor[1]][neighbor[0]].status = 5  # Moved Path

                self.visited_cells.append(neighbor)

    return False
```

Hình 8. Code thuật toán DFS

(thuật toán ucs và bidirectional search)

PHẦN 4: THỰC NGHIỆM, ĐÁNH GIÁ, PHÂN TÍCH KẾT QUẢ

4.1) Xây dựng giao diện

Khi chạy thực thi file **main.py** sẽ ra cửa sổ gồm một bản đồ ma trận có kích thước 41 x 41, các nút chức năng bên dưới bản đồ và banner cùng hướng dẫn thao tác góc phải.



Hình 9. Giao diện phần mềm khi bắt đầu

Để xây dựng giao diện như trên, đầu tiên nhóm chúng em khai báo các hằng như kích thước ma trận, tiêu đề project, mã màu (định dạng RGB),… Các hằng khai báo trong file tên (.....)và được liệt kê ở bảng bên dưới:

THÔNG TIN CÁC HẰNG		
STT	**Hằng**	**Giải thích**
1	WIDTH = 615	Kích thước bản đồ ma trận
2	SIZE = 700	Kích thước cửa số chương trình
3	SQUARE_SIZE = 15	Kích thước từng ô trong bản đồ
4	ROWS = 13	Số dòng trong bản đồ
5	COLS = 21	Số cột trong bản đồ

6	TITLE = " Artificial Intelligent Project - Group 9 "	Tiêu đề chương trình
7	FONT_NAME = “Verdana”	Tên font chữ mặc định
8	FONT_SIZE = 15	Kích thước font mặc định
9	GREY = (250, 250, 250)	Màu xám trắng, màu background của chương trình
10	GREEN = (16, 224, 92)	Màu xanh lá, dùng để chỉ định nút bắt đầu
11	RED = (224, 18, 70)	Mau đỏ, dùng để chỉ đinh nút kết thúc
12	PURPLE = (128, 0, 128)	Màu tím, đường đi từ nút bắt đầu đến nút kết thúc
13	BLACK = (0, 0, 0)	Màu đen, các nút vật cản không thể đi vào
14	PINK = (255, 179, 253)	Màu hồng, các nút đã xem xét xong
15	BLUE = (1, 255, 255)	Màu xanh lam, các nút hiện đang xem xét bởi thuật toán
16	DARK_GREY = (128, 128, 128)	Màu xám đen, màu của lưới ma trận

Bảng 1. Thông tin các hằng

Sau khi đã có các hằng, nhóm em thiết lập các hàm dùng đế lấy vị trí, xác định nút đó là gì và cài đặt từng nút theo thác tác của người dùng. Các hàm được khai báo trong lớp Node và cài đặt trong file **node.py**

Tiếp theo, nhóm em tiến hành vẽ lưới ma trận, vẽ các nút và ghi hướng dẫn thao tác trên giao diện cho người dùng. Các hàm được khai báo trong lớp Gui, được khai báo trong file **gui.py** và được gọi tại 1 hàm chung tên draw(). Hàm draw() này sẽ gọi lần lượt các hàm vẽ lưới ma trận, vẽ nút điều khiển, vẽ banner và hướng dẫn sử dụng. Tại giao diện chính có 6 nút chức năng được liệt kê tại bảng bên dưới:

THÔNG TIN CÁC NÚT CHỨC NĂNG		
STT	**Nút chức năng**	**Ý nghĩa**
1	A* Search	Chọn giải thuật A Star
2	BFS	Chộn giải thuật Breath First Search
3	DFS	Chọn giải thuật Depth First Search
4	Open map	Mở hộp thoại chọn file để chọn bản đồ
5	Save map	Mở hộp thoại chọn file để lưu bản đồ
6	Clear path	Xóa đường đi vừa tạo

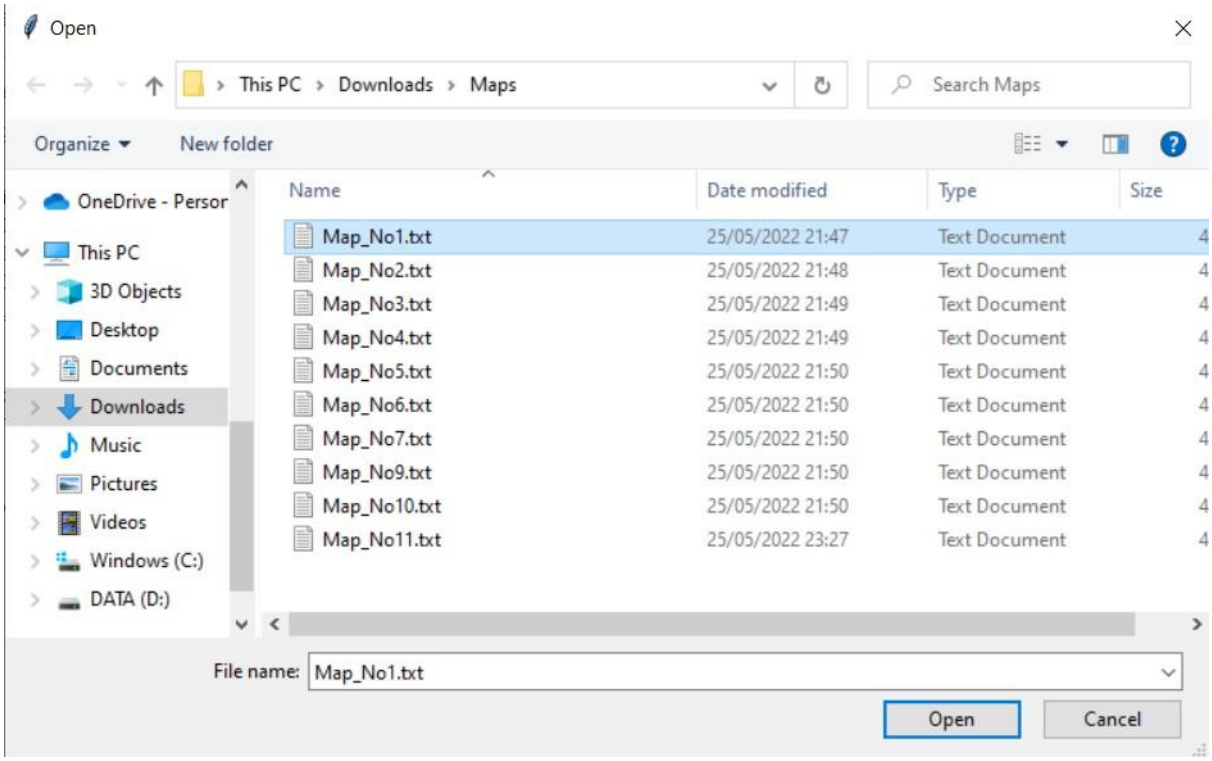
Bảng 2. Thông tin các nút chức năng

Để có thể lấy dữ liệu bản đồ từ file .txt, nhóm chúng em đã xây dựng hàm đọc và ghi file tại file **file_handling.py**. Tại file này, chúng em sử dụng thư viện tkinter và tính năng filedialog như đã đề cập phía trên để tương tác với file.

Bên cạnh đó, các loại giải thuật cũng được đặt theo tên file tương ứng như **A_Star.py**, **BFS.py**, **DFS.py** và đặt trong thư mục có tên **SearchAlgorithms.**

4.2) Hướng dẫn thực thi phần mềm

Bước 1: Bấm chọn nút Open map để mở hộp thoại chọn file và sử dụng bản đồ có sẵn



Hình 10. Hộp thoại chọn file của tkinter

Bước 2: Bấm chọn 1 trong 3 thuật toán (A*, BFS, DFS)

Bước 3: Nhấn phím Space để run thuật toán

Ngoài ra, ta có thể tùy chỉnh bản đồ bằng chuột:

+ Click chuột trái để tạo thêm các ô vật cản

+ Click chuột phải để xóa các ô, có thể dùng chuột phải để xóa ô vật cản, ô bắt đầu, hoặc ô kết thúc.

Khi thuật toán đã chạy xong, ta bấm nút Clear path hoặc nút “d” để xóa đường đi vừa tạo, thay đổi thuật toán khác để có thể so sánh chi phí, thời gian và số lượng nút đã duyệt.

Ta cũng có thể tạo một bản đồ mới và ghi nó vào file với nút Save map.

4.3) Kết quả sau khi chạy thuật toán

Sau khi đã mở file thành công, ta chọn 1 thuật toán để chương trình chạy (trong hình là thuật toán A Star)

Hình 11. Chọn thuật toán để bắt đầu thực thi

Ta bấm phím Space và quan sát thuật toán tìm đường, với mỗi ô đang xét, ô sẽ có màu xanh lam và khi thuật toán đã xét xong, ô sẽ có màu hồng. Màu tím là màu đường đi từ ô bắt đầu đến ô kết thúc.

Hình 12. Thuật toán đã thực hiện tìm đường hoàn tất

Với thuật toán A Star phía trên và bản đồ Map_No1.txt, ta thấy thuật toán mất thời gian là 1,7 giây để hoàn tất, chi phí đường đi là 83 nút và số lượng nút đã duyệt là 171.

4.4) So sánh các thuật toán

4.4.1) Sử dụng 1 Map_No1

Ta chọn Map_No1.txt là bản đồ ma trận để thực hiện 3 thuật toán được cài đặt.

Kết quả chạy 3 thuật toán Astar, BFS, DFS được thể hiện lần lượt như các hình dưới đây:

Hình 13. Thuật toán Astar trên Map_No1.txt

Hình 14. Thuật toán Breadth First Search trên Map_No1.txt

Hình 15. Thuật toán Depth First Search trên Map_No1.txt

Kết quả được tóm tắt qua bảng dưới đây

	Chi phí đường đi (nút)	**Thời gian (giây)**	**Số nút đã duyệt (nút)**
A*	83	1,7	171
BFS	83	5,68	476
DFS	83	2,59	204

Bảng 3. So sánh 3 thuật toán trên Map_No1.txt

Ta thấy giải thuật A Star có cả 3 thông số vượt trội so với 2 thuật toán kia, về mặt thời gian và chi phí. Trong khi thời gian cho thuật toán A Star là 1,7 giây thì BFS gấp hơn 3 lần là 5,68 giây và số lượng nút duyệt cũng nhiều hơn hẳn. Do tìm kiếm theo chiều rộng nên BFS đi qua rất nhiều nút và số lượng nút duyệt là 476, nhiều nhất trong 3 thuật toán.

4.4.2) Sử dụng nhiều map và tính trung bình

Ta tiếp tục so sánh 3 thuật toán dựa trên 3 thông số là thời gian, chi phí và số nút đã duyệt. Ở đây, ta sẽ sử dụng dữ liệu của 10 bản đồ từ Map_No1 đến Map_No10 và lấy trung bình thông số theo từng thuật toán để so sánh kết quả.

Kết quả chạy 3 thuật toán Astar, BFS, DFS trên 10 bản đồ được thể hiện lần lượt như các hình dưới đây:

Bảng 4. So sánh 3 thuật toán trên 5 maps

Theo các thông số trung bình, ta thấy giải thuật A* vẫn nhanh nhất vì đây là giải thuật tìm kiếm có thông tin. Thuật toán này chỉ tốn trung bình 2,888 giây để ra lời giải, nhanh gấp 2,7 lần so với BFS và gấp 1,8 lần so với DFS. Mặc dù ở Map_No3, A* chậm hơn DFS 0,64 giây nhưng các map còn lại, đặc biệt là Map_No5, A* lại nhanh hơn DFS đến 7,67 giây. BFS vẫn là giải thuật tốn nhiều thời gian nhất vì nó vét cạn không gian theo chiều rộng để tìm ra lời giải.

Chi phí đi từ điểm đầu đến đích cũng là một điểm cần lưu ý. A* có chi phí ngang với BFS nhưng tốn ít chi phí hơn DFS. Chi phí của DFS tại một số map bằng với BFS và A* nhưng tại một số map, đặc biệt là Map_No5, DFS gấp hơn 8 lần, khiến nó trở nên kém hiệu quả.

Số lượng nút đã duyệt trung bình cho thấy BFS đã xét rất nhiều nút (trung bình 728 nút) để tìm ra đường đi cuối cùng. Điều này phù hợp với lý thuyết vì đây là thuật toán tìm kiếm theo chiều rộng, nó tìm hết tất cả các trường hợp có thể xảy ra để đi đến mục tiêu. Ngược lại, một thuật toán có định hướng như A* sử dụng hàm Heuristic sẽ có số lượng nút duyệt ít nhất vì tại mỗi nút thuật toán này sẽ xem xét khoảng cách từ nút hiện tại đến nút đích và sẽ đi qua những nút nào có khoảng cách đang xét ngắn hơn.

PHẦN 5: PHẦN KẾT LUẬN

5.1) Đánh giá những kết quả đã thực hiện được

- Về mặt giải thuật:

+ Đã hoàn thành triển khai thực hiện các thuật toán BFS, DFS và A* ứng dụng chúng vào chương trình

+ Làm rõ được sự khác nhau giữa các thuật toán và so sánh chúng một cách trực quan thông quan chương trình đã xây dựng.

- Về mặt ứng dụng:

+ Đã hoàn thiện chương trình sử dụng Pygame để mô tả các thuật toán

+ Người học có thể tự tay xây dựng bản đồ phù hợp với nhu cầu

5.2) Định hướng phát triển

- Chương trình tìm kiếm đường đi của nhóm chúng em có tiềm năng phát triển thành một ứng dụng Game tương tác tìm kho báu theo định hướng giáo dục về giải thuật. Người chơi sẽ có khả năng tùy chỉnh độ khó từng màn chơi với số vật cản tỉ lệ thuận với độ khó được chọn. Tại mỗi màn chơi, người chơi sẽ di chuyển thủ công nhân vật mình để đi từ điểm đầu đến đích. Chương trình sẽ dựa vào đường đi của người chơi để phân tích hiệu quả, đồng thời so sánh đường đi đó với các giải thuật cài sẵn. Tiêu chí so sánh dựa trên thời gian, chi phí, quãng đường. Từ những thông số được so sánh, người dùng sẽ hiểu thêm về cách thức hoạt động của thuật toán và cách tư duy tìm đường dựa trên giải thuật tương ứng.

- Để đạt được định hướng đề ra, nhóm chúng em có những phương án như:

+ Xây dựng thêm nhiều thuật toán để chương trình đa dạng hơn

+ Cài đặt tính năng tự động tạo bản đồ tùy vào lựa chọn độ khó của người chơi. Mở rộng phạm vi bản đồ, giúp người chơi phân biệt được ưu nhược điểm của từng phương pháp trong phạm vi lớn.

+ Nâng cấp thêm các chế độ như : chạy từng bước bằng tay, hiển thị các giá trị của biến để người chơi có thể dễ dàng hiểu hơn

+ Cải thiện giao diện thân thiện với người chơi, có thể tích hợp các âm thanh tương ứng với sự kiện trong chương trình.

TÀI LIỆU THAM KHẢO

1. Pygame documentation, https://www.pygame.org/docs/ref/display.html
2. Tkinter documentation, https://tkdocs.com/
3. Đọc ghi file với thư viện Tkinter, https://www.pythontutorial.net/tkinter/tkinter-open-file-dialog/
4. Thuật toán A*, https://www.geeksforgeeks.org/a-search-algorithm/
5. Thuật toán BFS, https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/
6. Thuật toán DFS, https://www.geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph/